Formalisme fonctionnel et architecture intégrée du modèle AAM O94.5

Dossier technique AAM

20 octobre 2026

Table des matières

1 Description et objet **1**

2 Principes normatifs et limites de preuve **2**

3 Architecture par briques primitives **2**

4 Chargement et intégration des modèles locaux **3**
4.1 Bundle Transformers / SmolLM2 3
4.2 Politique d'orchestration du chat 3

5 Observation multimodale outillée **3**
5.1 Pipeline 3

6 Programmation compositionnelle et compilation **3**

7 Récompense, punition et adaptation causale **4**

8 Pipeline unifié chat–mémoire–RAG–compétences **4**

9 Commandes de validation **5**

10 Résultats et interprétation **5**

11 Conséquences et conclusions **5**

12 Réconciliation de validation et statut de version **5**
12.1 Dépendances et capacités encore à qualifier 5
12.2 Historique des corrections de test 6
12.3 Conclusion 6

1 Description et objet

O94.5 est un incrément de correction d'O94.4, dirigé par les défauts observés dans le chargement local des modèles, l'observation outillée, la compilation compositionnelle et la causalité du système de récompense/punition. Il ne rebaptise pas des fonctions déclaratives comme des capacités acquises : chaque affirmation opérationnelle doit être reliée à un artefact, une commande, une sortie et un cas de régression. Les sources de référence héritées, y compris `source_material/source_archive/sources_AAM_17.zip` et le fichier principal `DDD_Base_theorie_des_empreintes_complete_order_5.tex`, sont conservées sans modification comme matériaux de filiation.

2 Principes normatifs et limites de preuve

R1. Une observation est un processus exécuté sur la donnée sélectionnée ; le rapport distingue acquisition, décodage, extraction, empreinte, interprétation et vérification indépendante.

R2. Une composition compile à partir d'un DSL validé en source Python puis en bytecode CPython. Les seuls appels autorisés sont des composants enregistrés ; les fragments de code dans l'entrée DSL ne sont jamais exécutés.

R3. Un checkpoint de modèle n'est accepté qu'avec architecture/configuration, tokenizer et fichiers de poids cohérents. Le nom ou l'extension d'un fichier ne suffit pas à établir son architecture.

R4. Une exécution réussie n'est pas une preuve de vérité. Le retour de l'outil, la vérification déclarative indépendante et l'évaluation de satisfaction sont trois signaux distincts.

R5. Toute rétroaction qui modifie un poids de préférence doit avoir un événement causal, un identifiant de tâche, une empreinte d'événement et une politique bornée. Les doublons sont ignorés.

R6. Les composants locaux ne se substituent pas silencieusement à un modèle explicitement choisi qui échoue. Les téléchargements de poids sont désactivés par défaut et le code distant n'est pas activé automatiquement.

R7. Les artefacts générés et le code du projet ne sont pas des bacs à sable. L'exécution est bornée par délai et taille de sortie ; une validation de sécurité d'environnement distincte reste requise.

3 Architecture par briques primitives

La séparation normative est la suivante : une *compétence* exécute une action typée ; une *méthode* ordonne des compétences et leurs données de sortie ; une *règle* décide une branche à partir de faits observés ou de critères explicites. Le runtime, le chat, l'observateur et le solveur réutilisent les mêmes contrats, les mêmes traces et la même politique de causalité.

Brique	Contrat fonctionnel	Preuve opérationnelle attendue
Observation	Lire des octets/événements accessibles, identifier le décodeur et conserver l'empreinte de source.	Rapport reproductible, outil réellement appelé, statut de décodage, hash d'entrée et journal causal.
Sonde	Produire un point de vue explicitement défini et différenciable.	Identité de la sonde, paramètres, résultat et test de non-duplication.
Empreinte	Représenter les caractéristiques observées et leur provenance sans confondre hash d'intégrité et représentation sémantique.	Statistiques, versions de transformation et validation de cohérence.
Compétence	Exécuter une opération avec contrat d'entrée/sortie et résultat structuré.	Statut, entrée/sortie hashées et effets réels rapportés.
Méthode	Enchaîner des compétences, transmettre les sorties et interrompre sur erreur.	Trace de chaque nœud et résultat différentiel attendu/obtenu.
Règle	Sélectionner branche ou action selon des conditions typées.	Conditions évaluées, branche retenue et contre-exemples.
Vérificateur	Tester des critères indépendants de préférence et de génération.	Identifiant du vérificateur, hash des critères, hash de la preuve et résultat PASS/FAIL.
Orchestrateur	Sélectionner voie, budget, mémoire et compétences selon demande/contexte.	Décision traçable, résultat, correction de préférence bornée et cause de l'étape suivante.

4 Chargement et intégration des modèles locaux

4.1 Bundle Transformers / SmolLM2

Le backend `transformers_local` accepte un répertoire Hugging Face complet ou un fichier de poids sélectionné dans ce répertoire. La sélection du seul `model.safetensors` résout le répertoire parent ; le système exige `config.json`, une famille de poids reconnue et des métadonnées/tokenizer local. `AutoTokenizer` et `AutoModelForCausalLM` reçoivent `local_files_only=True` et `trust_remote_code=False`. Les empreintes SHA-256 de tous les fichiers inventoriés sont calculées en lecture par blocs à l'enregistrement et vérifiées avant le premier chargement. Si l'inventaire change, le modèle doit être réenregistré de façon explicite.

Les formats `.pt`/`.pth` ne sont pas une architecture. Si un fichier identifié comme state-dict est accompagné de la configuration et du tokenizer d'un modèle Transformers, un chargement contrôlé avec `weights_only=True` est tenté, suivi d'un chargement strict de tous les paramètres. Le code refuse le chargement pickle non sûr, le mismatch de clés et les checkpoints TorchScript mal classés. Les artefacts TorchScript ont leur backend tensoriel distinct. L'entraînement LoRA requiert les dépendances optionnelles, un modèle compatible, une liste de modules cibles reconnue/validée et un corpus ayant une séparation train/holdout. La création de paires supervisées n'est pas présentée comme un entraînement des poids.

4.2 Politique d'orchestration du chat

Le modèle sélectionné explicitement conserve la priorité. En mode automatique, le runtime peut choisir le modèle intégré le plus récemment enregistré et dont le bundle est présent ; l'échec est exposé à l'utilisateur et ne déclenche pas de substitution silencieuse. Le prompt envoyé au générateur contient la question et le contexte RAG pertinent comme données séparées. Le RAG conserve la provenance ; ni citation ni fluidité n'est considérée comme preuve de vérité.

5 Observation multimodale outillée

5.1 Pipeline

1. Valider le chemin, refuser les liens symboliques et les objets qui ne sont pas des fichiers ordinaires pour le chemin documentaire.
2. Calculer SHA-256 en lecture par blocs, taille, horodatage et identifiant du format.
3. Sélectionner le décodeur en fonction du format et inventorier les outils locaux réellement disponibles.
4. Appeler les outils détectés : Poppler pour PDF, Tesseract pour OCR, FFprobe/FFmpeg pour médias, parseur XML ZIP standard pour DOCX. Les PDF sans couche de texte sont rasterisés avec `pdftoppm` et OCRisés par Tesseract, avec un plafond de huit pages par passage.
5. Émettre un rapport sans hash circulaire, écrire la somme SHA-256 dans un sidecar, puis calculer une empreinte AAM supplémentaire sur le texte extrait si disponible.
6. En option, envoyer le texte dérivé au RAG avec un fichier de provenance ; la source originale reste en lecture seule.

Une vidéo produit des métadonnées FFprobe et des images échantillonnées selon un budget ; l'audio produit métadonnées et caractéristiques acoustiques sur une fenêtre bornée. Une transcription est déclarée seulement si un modèle ASR local a été configuré. Le rapport n'affirme pas une compréhension sémantique, une identification universelle d'objet ou une reproduction exacte sans tests séparés.

6 Programmation compositionnelle et compilation

Le compilateur O94.5 accepte un JSON versionné : `invoke`, `sequence`, `condition` et `map`. Les expressions de données autorisent uniquement références de variables, littéraux, listes/objets et opé-

rateurs purs bornés. La profondeur du graphe, le nombre de nœuds et les listes itérées sont plafonnés pour éviter une explosion accidentelle de ressources.

Le pipeline est : parse JSON, validation du schéma, vérification des IDs de composants dans le catalogue approuvé, émission déterministe de source Python, compilation CPython, production de bytecode, relecture AST et écriture d'un manifeste liant hash du DSL normalisé, source, bytecode et catalogue des composants. L'exécution de l'artefact vérifie le hash source et le manifeste, exige une autorisation explicite et applique délai/caps stdout-stderr. Elle tourne dans les permissions du compte : ce contrôle de lancement n'est pas un bac à sable du système d'exploitation.

L'atelier de code est un outil réel associé à ces briques : liste de fichiers, ouverture/sauvegarde atomique, détection de conflits de modification externe, historique de sauvegarde, recherche/remplacement, insertion/suppression de ligne, analyse AST, compilation Python/bytecode et exécution avec confirmation. Il ne s'agit pas d'affirmer que toute application arbitraire peut être synthétisée sans spécification, test ou débogage.

7 Récompense, punition et adaptation causale

La politique sépare trois dimensions : préférence utilisateur r_u, fiabilité opérationnelle r_e, et résultat vérifié sur critères explicites r_v. L'ajustement de priorisation est borné :

$$A(c) = \mathrm{clip}_{[-2,2]}(2r_u(c) + 0.5r_e(c) + r_v(c)).$$

Chaque dimension met à jour sa propre moyenne exponentielle :

$$r' = (1-\alpha)r + \alpha s, \qquad 0.02 \le \alpha \le 0.5, \quad s \in [-1, 1].$$

Un statut d'exécution `PASS_BOUNDED` indique seulement que la voie a fonctionné sous son contrat. Un événement `UNRESOLVED` n'entraîne pas une pénalité automatique : absence de solution prouvée n'est pas preuve d'erreur. Une vérification indépendante exige le hash des critères, des éléments évalués, l'ID du vérificateur et un événement causal. Le hash d'événement empêche le double-apprentissage d'un même signal. Le score ajuste la sélection future de compétences ; il ne modifie pas directement les règles de vérification et ne peut pas falsifier les métriques.

8 Pipeline unifié chat–mémoire–RAG–compétences

1. Enregistrer la requête et son empreinte ; mettre en pause les travaux autonomes concurrents qui partagent les mêmes ressources.
2. Observer requête, pièces jointes et état ; décider un mode et une sélection de compétence sous une politique visible.
3. Si autorisé, chercher dans le RAG, classer les enregistrements pertinents, conserver les citations et construire le contexte de génération.
4. Appeler la compétence ou le modèle choisi ; ne pas substituer silencieusement un générateur sur échec.
5. Évaluer l'état opérationnel, enregistrer les événements de fin et leur parenté causale ; si un vérificateur explicite est fourni, actualiser le signal de validation indépendamment.
6. Exposer réponse, sources, modèle/provider, route, état de vérification et diagnostics dans l'interface.
7. Reprendre l'autonomie selon budget et contrôle de l'utilisateur ; persister les résultats, les préférences bornées et le point de reprise.

9 Commandes de validation

Depuis la racine du modèle, utiliser l'environnement Python compatible avec le manifeste de dépendances :

```
python -m py_compile aam_o93/model_integration.py aam_o93/observation_tools.py aam_o93/code_wor
pytest -q tests/test_o945_transformers_bundle.py tests/test_o945_composition_compiler_workbench
python scripts/preflight_o945.py
```

Pour charger SmolLM2, installer d'abord `requirements-transformers.txt` dans un environnement disposant des dépendances et fournir le répertoire local complet issu du téléchargement autorisé. Le test d'un fixture synthétique n'est pas une preuve d'inférence de SmolLM2.

10 Résultats et interprétation

Les résultats effectifs sont consignés dans `RAPPORT_TESTS_O94_5_FR.md`, dans les reçus JSON de `receipts/O94_5/` et les journaux associés. Les valeurs ci-dessus décrivent le contrat conçu ; seul un reçu de test daté autorise une revendication de fonctionnement. Les dépendances non installées, modèles absents et modalités sans décodeur restent des limites d'environnement qualifiées, pas des conclusions de potentiel.

11 Conséquences et conclusions

Conceptuellement, O94.5 sépare « observer », « représenter », « composer », « compiler », « exécuter » et « vérifier », afin qu'une trace ne soit pas confondue avec une compétence généralisée. Opérationnellement, chaque capacité produit une sortie et une provenance et permet des tests différenciés. Le statut de la version est défini par la checklist séquentielle et les preuves incluses dans la livraison ; aucune promesse de généralité totale n'est faite par anticipation.

12 Réconciliation de validation et statut de version

La campagne complète finale du 20 octobre 2026 a rendu `PASS_BOUNDED`, code de retour nul, **419 tests réussis, 0 échec, 5 avertissements et 21 sous-tests réussis** en 199,38 secondes sous Xvfb. Le reçu machine `receipts/O94_5/P6_full_suite_regression_20261020T114952Z_19680.json` porte le SHA-256 du journal complet. Les reçus ciblés sont regroupés dans `registre_prouves_O94_5.json` et `O94_5_TEST_MATRIX.json`.

Les preuves réelles locales couvrent les appels Tesseract, Poppler et FFmpeg/FFprobe ; l'atelier compositionnel compile du JSON DSL vers Python/bytecode et les scénarios vérifient le diagnostic d'un projet, l'exécution de composants approuvés et le flux d'édition multi-onglets. La récompense/punition conserve trois dimensions et lie les changements aux événements de runtime. La conversation relie génération, RAG, pièce jointe et mémoire dans le parcours de runtime testé.

12.1 Dépendances et capacités encore à qualifier

Les poids du SmolLM2-360M de l'utilisateur, son tokenizer/configuration et le runtime Transformers ne sont pas présents dans l'environnement de construction. Les tests valident l'inspection/enregistrement des bundles, l'intégrité, la politique de chargement local et le dispatch générateur via double d'API ; ils ne sont pas une inférence sur les vrais poids SmolLM2. L'incident est maintenu ouvert dans le registre environnemental jusqu'à la réalisation d'un test sur la machine possédant les fichiers. L'ASR réel est pareillement déclaré uniquement si un moteur et ses poids sont effectivement configurés.

12.2 Historique des corrections de test

Les campagnes intermédiaires ont montré des problèmes de lancement X, une cascade de contrôle SHA-256 liée à la réécriture de reçus pendant pytest, et deux assertions qui n'acceptaient pas le statut contractuel `PASS_BOUNDED`. Chaque incident est conservé dans les registres. Le runner actuel crée des artefacts uniques par tentative puis actualise les alias stables après sortie du processus ; les deux assertions ont été alignées sur le contrat sans affaiblir les contrôles du décodage, de la source ou du rapport.

12.3 Conclusion

La phase P6 est `COMPLETE_EVIDENCED` ; P7 formalise, documente et rend la sauvegarde restaurable ; P8 reste ouverte jusqu'à l'extraction et la vérification CRC/SHA des trois archives de livraison. Le statut `PASS_BOUNDED` établit uniquement le comportement observé dans le périmètre et le budget des tests correspondants. Il ne signifie ni généralité universelle, ni réussite anticipée, ni limitation anticipée du potentiel du modèle.